

近年来强化学习更多地应用到了大模型后训练上，也就是把大模型当成 agent，通过其与环境交互获得奖励，用策略梯度来优化 LLM 的生成，但 LLM RL 对比传统 RL 而言动作空间更大，rollout 分支更多，语义更复杂，有很多新的问题。

那么这里先简述一下问题，我们通过这个梯度公式来使得 LLM 的策略获得最大的累计回报，但是在稀疏奖励和复杂语义的背景下估计优势 A_t 比较困难，本工作以及所提及的相关工作都聚焦于如何更好地设计这里的 A_t

近年来一个很主流的算法是 GRPO，通过群组比较的方法，它省去了独立的价值网络，通过某一条回答的奖励与组内平均奖励的差值来估计本条回答的优势。但可以注意到它只为每一条回答分配一个奖励，并把这个优势广播到这条回答的所有 token，是粗糙的轨迹级粒度，那么随着应用从单轮问答扩展到多轮交互，轨迹变长，奖励稀疏问题更突出，细粒度的 RLVR 算法应运而生。

相关工作方面，除了 GRPO 以及相关变体，还有步骤级别的优势估计，比如 GiGPO，如果说 GRPO 是把同一问题 rollout 的所有轨迹放在一起比较优势，那么 GiGPO 就是把状态相同（就是这里的 s 把相同），但做出了不同动作的分叉统计起来，根据不同分叉的结果来给每个分叉动作打分，得到步骤级优势，而 iStar 就是单独训练了一个过程奖励模型给动作打分。

词元级别的优势估计包括 HEPO、TEMPO 等等，HEPO 发现 LLM 生成的词元中只有少量是不确定性高，也就是熵值比较高的、具有决策意义的 token，因此有一个熵掩膜只优化高熵的 token，TEMPO 就是跟 GiGPO 思路比较像，用一个前缀树，统计树叶上的最终奖励来得到每一个 token 节点的优势，是一个 token 级别的优化。

还有分层的优势估计比如 ArCher，它为动作步设计了价值网络 Q 和 V ，来估计动作的优势，但是这个动作步的优势其实也是广播到了动作内的每个 token 上，其实不算真正的嵌套，因此...右边那一栏字

接下来我的研究方案分为三个主要部分，第一个是复现经典的算法，分别包括步骤级别如 GiGPO、ARPO 等，词元级别如 TAPO、TEMPO 等，作为后续实验的一个组件。然后是调和方式的对比，更菜我们说我们主要解决的问题是如何更好地得到优势 A ，那么现有的方法分别可以提供步骤级别的 A_{step} 和词元级别的 A_{token} ，调和就是思考用什么方式把他们融合到一起，比较简单的两种是加法调和和乘法调和，像这里的两个图所示，加法就是简单线性相加， λ 是超参数；

乘法是步骤级别的优势乘以词元级别的熵门控，得到最终的嵌套优势

还有一个比较有挑战性的想法就是动作隐向量引导的双粒度优势估计，我的想法是给“动作/步骤”这个层级单独安排一个隐向量，它蕴含的语义代表着动作本身，面对一个新状态我们先读取状态生成动作隐向量，然后把这个向量作为新的嵌入指导后续的 token 生成，对隐向量和 token 分别进行策略梯度优化，实现嵌套算法。

最后是消融实验，也就是在完整的 **NEST** 下，失能其中的某一个粒度的信号去研究该粒度的作用和两个粒度的相互作用。我预期的结果就是可以实现一个轻量化的嵌套优势估计并且能说明两个粒度的优势估计是可以很好地互补的。

进度安排

感谢